

PORTFOLIO

แฟ้มสะสมผลงานประกอบการสัมภาษณ์

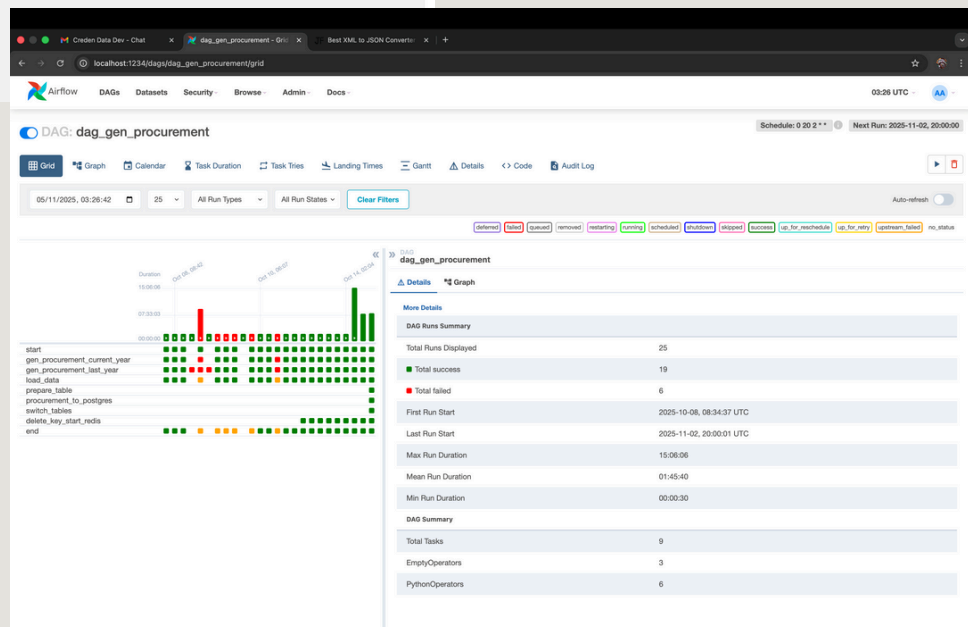
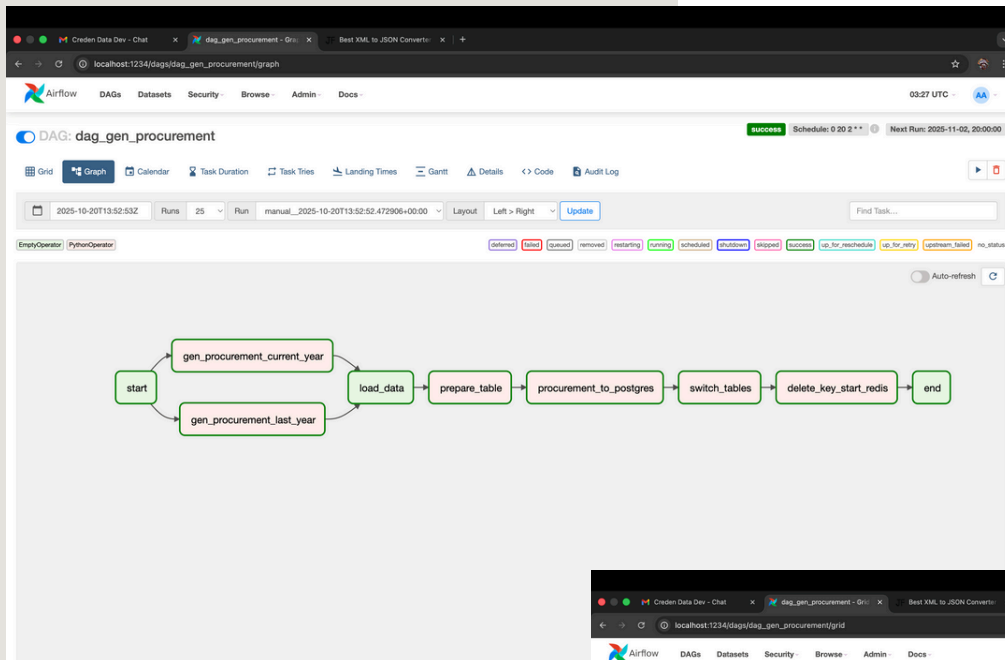
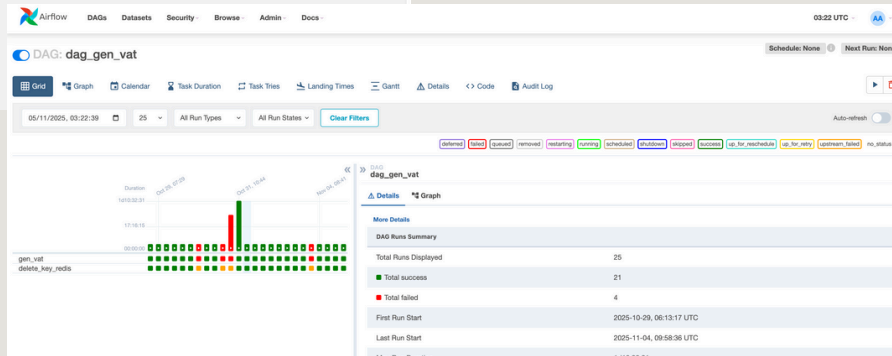
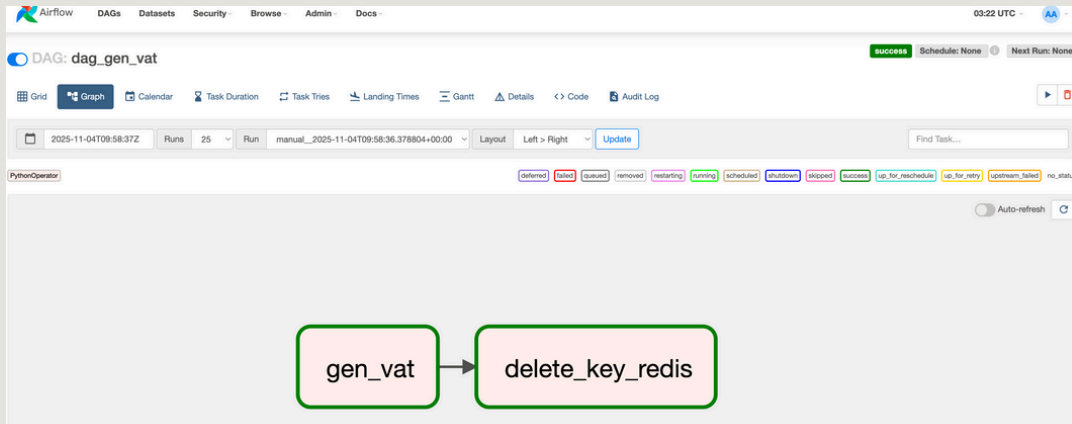


นายภูวดล วรบุตร

วิทยาศาสตร์บัณฑิต

มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ

Apache Airflow



```

        start += 1000
        time.sleep(3)
    ret["success"] = True if (total_keep_log + total_contract + total_new) > 0 else False
    print(f"Finished processing total records: Keep Log={total_keep_log}, Contract={total_contract}, New={total_new}")

```

```
finally:
    pg_conn.commit()
    cursor.close()
    pg_cursor.close()
```

```
ENV_SERVER = get_env('ENV_SERVER')
USER_MAIL = get_env("USER_MAIL")
USER_MAIL_LIST = ast.literal_eval(USER_MAIL)
```

```
if ENV_SERVER == 'DEV':
    user_sent_mail = USER_MAIL_LIST[0]
else:
    user_sent_mail = USER_MAIL_LIST

notify_success_log_with_mail = partial(notify_success_log, USER_MAIL=user_sent_mail)
notify_failure_log_with_mail = partial(notify_failure_log, USER_MAIL=user_sent_mail)
```

```
year_current = int(datetime.now().strftime("%Y"))+543
year_last = int(datetime.now().strftime("%Y"))+542
```

```
def gen_data_procurement_process(year, **kwargs):
    start_process = start_run_process(year)
    if year == 2567 and start_process == 0:
        start_process = 1
    gen_data_procurement(year, start_process, **kwargs)
    return 'gen_data_procurement Success'
```

```
def delete_redis():
    process_sucess_delete_redis(year_current)
    process_sucess_delete_redis(year_last)
    return 'delete key redis Success'
```

```
default_args = {
    'owner': 'aut',
    'depends_on_past': False,
    'email': None,
    'email_on_failure': True,
    'email_on_retry': False,
    'email_on_success': True,
    'retries': 10,
    'retry_delay': timedelta(seconds=30)
}
```

```
with DAG(
    dag_id="dag_gen_procurement",
    start_date = datetime(2025, 9, 14),
    default_args = default_args,
    schedule_interval = '0 20 2 * *',
    catchup = False,
    max_active_runs=1,
) as dag:
    start = EmptyOperator(task_id="start")
    load = EmptyOperator(task_id="load_data")
    end = EmptyOperator(task_id="end")

    task_1 = PythonOperator(
        task_id="gen procurement current year",
```

```
with DAG(
    dag_id="dag_gen_procurement",
    start_date = datetime(2025, 9, 14),
    default_args = default_args,
    schedule_interval = '0 20 2 * *',
    catchup = False,
    max_active_runs=1,
) as dag:
    start = EmptyOperator(task_id="start")
    load = EmptyOperator(task_id="load_data")
    end = EmptyOperator(task_id="end")
```

```
task_1 = PythonOperator(
    task_id="gen_procurement_current_year",
    python_callable=gen_data_procurement_process,
    op_args=[year_current],
    on_success_callback=notify_success_log_with_mail,
    on_failure_callback=notify_failure_log_with_mail,
    dag=dag
)
```

```
task_2 = PythonOperator(
    task_id="gen_procurement_last_year",
    python_callable=gen_data_procurement_process,
    op_args=[year_last],
    on_success_callback=notify_success_log_with_mail,
    on_failure_callback=notify_failure_log_with_mail,
    dag=dag
)
```

```
task_3 = PythonOperator(
    task_id="prepare_table",
    python_callable=prepare_table,
    on_success_callback=notify_success_log_with_mail,
    on_failure_callback=notify_failure_log_with_mail,
    dag=dag
)
```

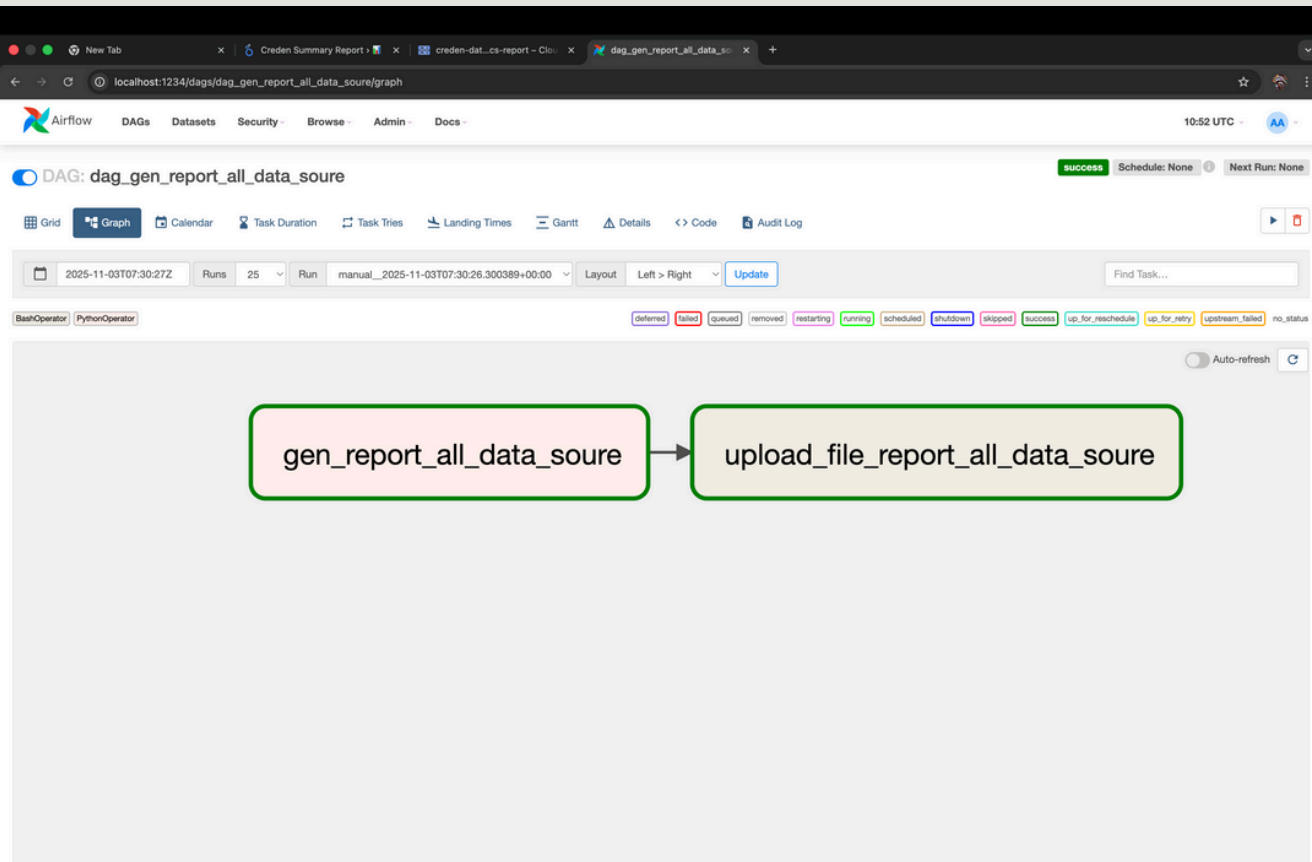
```
task_4 = PythonOperator(
    task_id="procurement_to_postgres",
    python_callable=procurement_to_postgres,
    on_success_callback=notify_success_log_with_mail,
    on_failure_callback=notify_failure_log_with_mail,
    dag=dag
)
```

```
task_5 = PythonOperator(
    task_id="switch_tables",
    python_callable=switch_tables,
    on_success_callback=notify_success_log_with_mail,
    on_failure_callback=notify_failure_log_with_mail,
    dag=dag
)
```

```
task_6 = PythonOperator(
    task_id="delete_key_start_redis",
    python_callable=delete_redis,
    dag=dag
)
```

```
start >> [task_1,task_2] >> load >> task_3 >> task_4 >> task_5 >> task_6 >> end
```


Automated Data Export from Database to GCS using Apache Airflow



The screenshot shows the Google Cloud Storage console. The 'creden-data-gcs-report' bucket is selected. The 'Folder browser' on the left lists various folders and files. The 'Objects' table on the right shows a list of files, including 'gen_report_all_data_soure.csv'.

Name	Size	Type	Created	Storage class	Last modified	Public
data-logs-cookie-script/	2.5 MB	text/csv	Jun 11, 2024, 3:54:53 PM	Standard	Jun 11, 2024, 3:54:53 PM	No
data-logs-cookie-script/	—	Folder	—	—	—	—
data_log_link_cookie.csv	1.8 MB	text/csv	Nov 5, 2025, 12:39:42 AM	Standard	Nov 5, 2025, 12:39:42 AM	No
dev_gen_report_all_data_soure.csv	5.9 KB	text/csv	Nov 3, 2025, 2:33:42 PM	Standard	Nov 3, 2025, 2:33:42 PM	No
dump-company-wh/	—	Folder	—	—	—	—
etl_history_payment_member.json	11.7 MB	application/json	Nov 5, 2025, 12:59:57 AM	Standard	Nov 5, 2025, 12:59:57 AM	No
etl_history_redeem_coupon.json	723.1 KB	application/json	Nov 5, 2025, 12:29:49 AM	Standard	Nov 5, 2025, 12:29:49 AM	No
etl_login_token_timebase.json	346.3 KB	application/json	Nov 5, 2025, 12:30:12 AM	Standard	Nov 5, 2025, 12:30:12 AM	No
etl_logs_company_partnership/	—	Folder	—	—	—	—
etl_member_timebase_center.json	264.1 KB	application/json	Nov 5, 2025, 12:30:08 AM	Standard	Nov 5, 2025, 12:30:08 AM	No
gen_report_all_data_soure.csv	5.9 KB	text/csv	Nov 3, 2025, 11:30:58 AM	Standard	Nov 3, 2025, 11:30:58 AM	No
logs_bot_dbd.csv	107.5 MB	text/csv	Mar 26, 2024, 12:43:28 AM	Standard	Mar 26, 2024, 12:43:28 AM	No
logs_bot_dbd_2.csv	33.1 MB	text/csv	Nov 5, 2025, 4:59:43 PM	Standard	Nov 5, 2025, 4:59:43 PM	No
member_register.json	152.5 MB	application/json	Nov 5, 2025, 12:33:17 AM	Standard	Nov 5, 2025, 12:33:17 AM	No
outfileamlo.csv	2.5 MB	text/csv	Jun 11, 2024, 4:05:06 PM	Standard	Jun 11, 2024, 4:05:06 PM	No
share_holder.csv	3.8 GB	text/csv	Jun 7, 2024, 2:18:20 PM	Standard	Jun 7, 2024, 2:18:20 PM	No
test.csv	2.7 MB	text/csv	Aug 18, 2023, 3:40:02 PM	Standard	Aug 18, 2023, 3:40:02 PM	No

```

import pandas as pd
from datetime import datetime
from modules.env import get_env
from pymongo import MongoClient

ENV_SERVER = get_env('ENV_SERVER')
HOST_MONGO = get_env('HOST_MONGO_APP')
DB_NEWDBD = "newdbd"
server = 'dev' if ENV_SERVER != 'PRODUCTION' else 'prod'
client_center = MongoClient(HOST_MONGO, connectTimeoutMS=60000, socketTimeoutMS=90000, serverSelectionTimeoutMS=60000)
db_newdbd = client_center[DB_NEWDBD]
col_company_wh = db_newdbd['company_wh']
col_vat_rd_new = db_newdbd['vat_rd_new']
col_procurement_new = db_newdbd['procurement_new']
col_share_holder_new = db_newdbd['share_holder_new']
col_web_open_api_dbd = db_newdbd['web_open_api_dbd']
col_history_order_document = db_newdbd['history_order_document']
col_financial_statements_detail = db_newdbd['financial_statements_detail']

current_year = datetime.now().year
start_of_year = datetime(current_year, 1, 1)

def order_fields_as_list(results):
    final_results = []
    for doc in results:
        ordered_doc = {
            "data_date": doc["data_date"],
            "count": doc["count"],
            "type": doc["type"]
        }
        final_results.append(ordered_doc)
    return final_results

def gen_data_report_from_collection(col_name, column_name, table_name_type):
    pipeline_company_wh = [
        {"$addFields": {"data_date": {"$toDate": f"${column_name}"}}},
        {"$match": {"data_date": {"$gte": start_of_year}}},
        {"$group": {
            "_id": {"$dateToString": {"format": "%Y-%m-%d", "date": "$data_date"}},
            "count": {"$sum": 1}
        }},
        {"$sort": {"_id": 1}},
        {"$project": {"_id": 0, "data_date": "$_id", "count": 1}},
        {"$addFields": {"type": table_name_type}}
    ]
    results = list(col_name.aggregate(pipeline_company_wh))
    data_results = order_fields_as_list(results)
    return data_results

def gen_file_report_data_soure():
    company_wh = gen_data_report_from_collection(col_company_wh, 'createDtm_ISODATE', 'company_wh')
    vat_rd_new = gen_data_report_from_collection(col_vat_rd_new, 'createDtm_ISODATE', 'vat_rd_new')
    web_open_api_dbd = gen_data_report_from_collection(col_web_open_api_dbd, 'fetchd_at', 'web_open_api_dbd')
    procurement_new = gen_data_report_from_collection(col_procurement_new, 'createDtm_ISODATE', 'procurement_new')
    share_holder_new = gen_data_report_from_collection(col_share_holder_new, 'createDtm_ISODATE', 'share_holder_new')
    financial_statements_detail = gen_data_report_from_collection(col_financial_statements_detail, 'create_isodate', 'financial_statements_detail')
    history_order_document = gen_data_report_from_collection(col_history_order_document, 'timestamp_create_order', 'history_order_document')

    all_data = (
        company_wh + vat_rd_new + web_open_api_dbd +
        procurement_new + share_holder_new +
        financial_statements_detail + history_order_document
    )
    df_all = pd.DataFrame(all_data)

import ast
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.operators.bash_operator import BashOperator
from functools import partial
from datetime import datetime, timedelta
from modules.new_email_airflow import notify_success_log, notify_failure_log
from modules.gen_report_all_data_soure import gen_file_report_data_soure
from modules.env import get_env

ENV_SERVER = get_env('ENV_SERVER')
USER_MAIL = get_env('USER_MAIL')
USER_MAIL_LIST = ast.literal_eval(USER_MAIL)
server = 'dev' if ENV_SERVER != 'PRODUCTION' else 'prod'

if ENV_SERVER == 'DEV':
    user_sent_mail = USER_MAIL_LIST[0]
else:
    user_sent_mail = USER_MAIL_LIST

default_args = {
    'owner': 'aut',
    'depends_on_past': False,
    'email': None,
    'email_on_failure': True,
    'email_on_retry': False,
    'email_on_success': True,
    'retries': 3,
    'retry_delay': timedelta(seconds=30)
}

with DAG(
    dag_id="dag_gen_report_all_data_soure",
    start_date = datetime(2025, 11, 1),
    default_args = default_args,
    schedule_interval=None,
    max_active_runs=1,
) as dag:

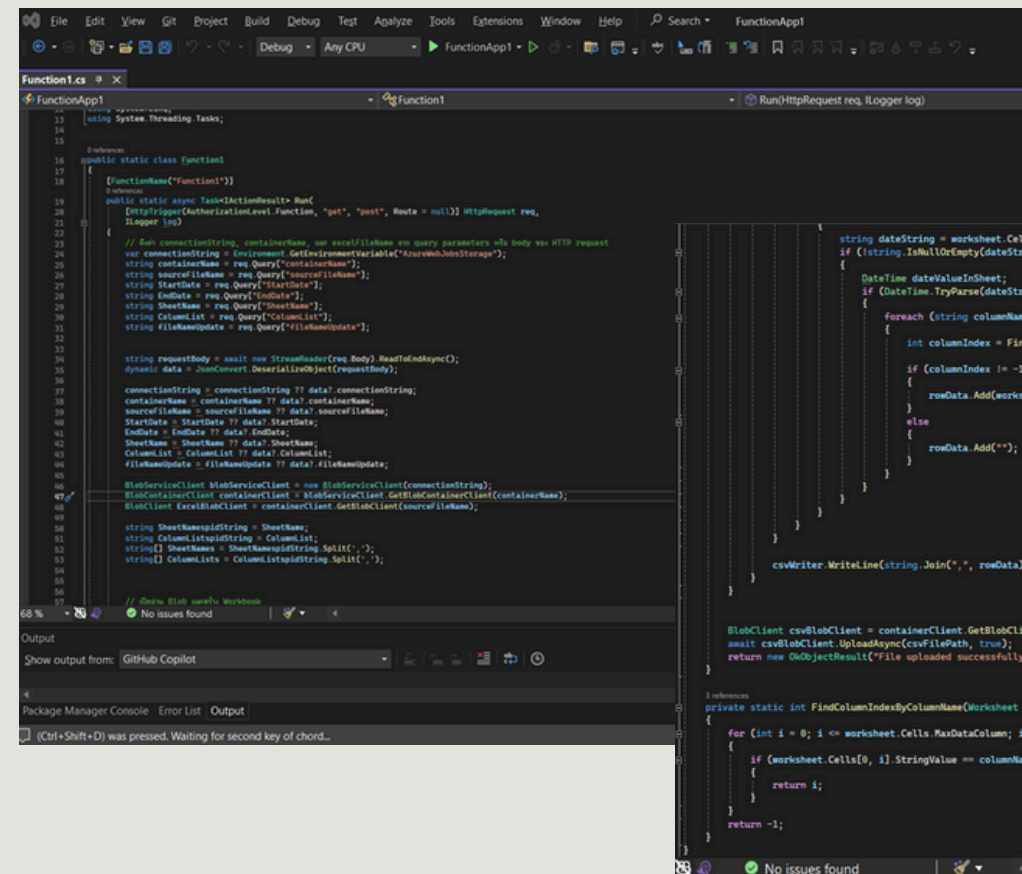
    gen_report_all_data_soure = PythonOperator(
        task_id="gen_report_all_data_soure",
        python_callable=gen_file_report_data_soure,
        on_success_callback=partial(notify_success_log, USER_MAIL=user_sent_mail),
        on_failure_callback=partial(notify_failure_log, USER_MAIL=user_sent_mail),
        dag=dag
    )

    upload_file_report_all_data_soure = BashOperator(
        task_id='upload_file_report_all_data_soure',
        bash_command=f'gsutil cp /tmp/{server}_gen_report_all_data_soure.csv gs://creden-data-gcs-report/Looker-Studio',
        dag=dag
    )

    gen_report_all_data_soure >> upload_file_report_all_data_soure

```

Create C# Script on Azure Functions for transform data to csv format files



```
using System.Threading.Tasks;

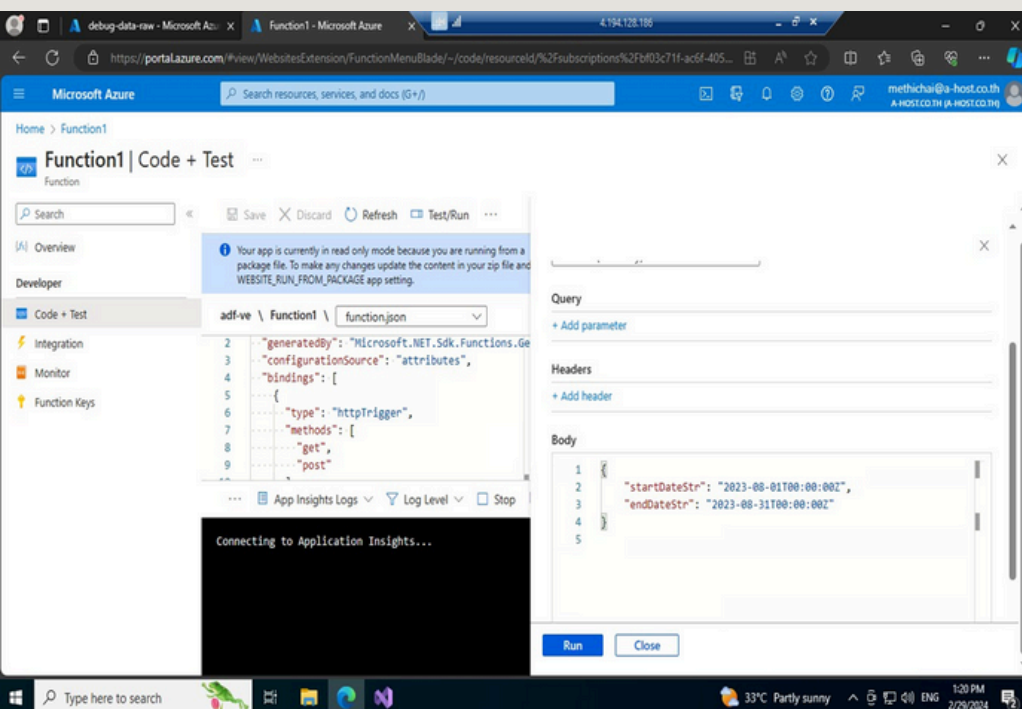
namespace FunctionApp1
{
    [FunctionName("Function1")]
    public static async Task Run(
        [HttpTrigger(AuthorizationLevel.Function, "get", "post", Route = null)] HttpRequest req,
        ILogger log)
    {
        // Run connection string, containerName, and excelFileName as query parameters who body was HTTP request
        var connectionString = Environment.GetEnvironmentVariable("AzureWebJobsStorage");
        string containerName = req.Query["containerName"];
        string sourceFileName = req.Query["sourceFileName"];
        string startDate = req.Query["startDate"];
        string endDate = req.Query["endDate"];
        string sheetName = req.Query["sheetName"];
        string columnList = req.Query["columnList"];
        string fileNameUpdate = req.Query["fileNameUpdate"];

        string requestBody = await new StreamReader(req.Body).ReadToEndAsync();
        dynamic data = JsonConvert.DeserializeObject(requestBody);
        connectionString = connectionString ?? data?.connectionString;
        containerName = containerName ?? data?.containerName;
        sourceFileName = sourceFileName ?? data?.sourceFileName;
        startDate = startDate ?? data?.startDate;
        endDate = endDate ?? data?.endDate;
        sheetName = sheetName ?? data?.sheetName;
        columnList = columnList ?? data?.columnList;
        fileNameUpdate = fileNameUpdate ?? data?.fileNameUpdate;

        BlobServiceClient blobServiceClient = new BlobServiceClient(connectionString);
        BlobContainerClient containerClient = blobServiceClient.GetBlobContainerClient(containerName);
        BlobClient excelBlobClient = containerClient.GetBlobClient(sourceFileName);

        string sheetNameUpdate = sheetName;
        string[] sheetNames = sheetNameUpdate.Split(",");
        string[] columnLists = columnList.Split(",");

        // Open the excel file
        // No issues found
    }
}
```



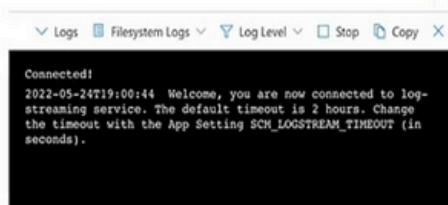
Function1 | Code + Test

Test/Run

Test input:

```
{
  "startDate": "2023-08-01T00:00:00Z",
  "endDate": "2023-08-31T00:00:00Z"
}
```

Run



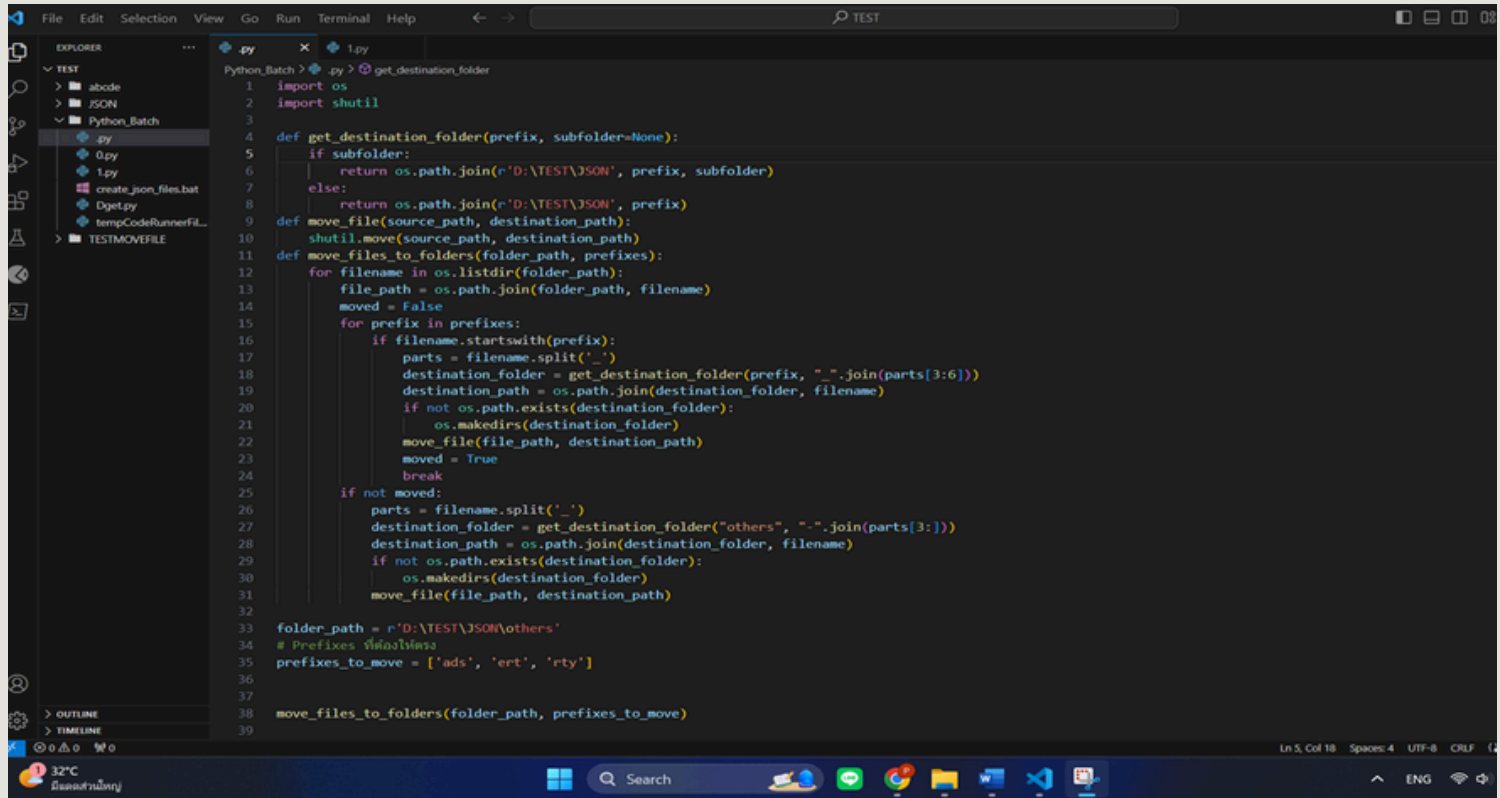
Logs

Connected!

2022-05-24T19:00:44 Welcome, you are now connected to log-streaming service. The default timeout is 2 hours. Change the timeout with the App Setting SCM_LOGSTREAM_TIMEOUT (in seconds).

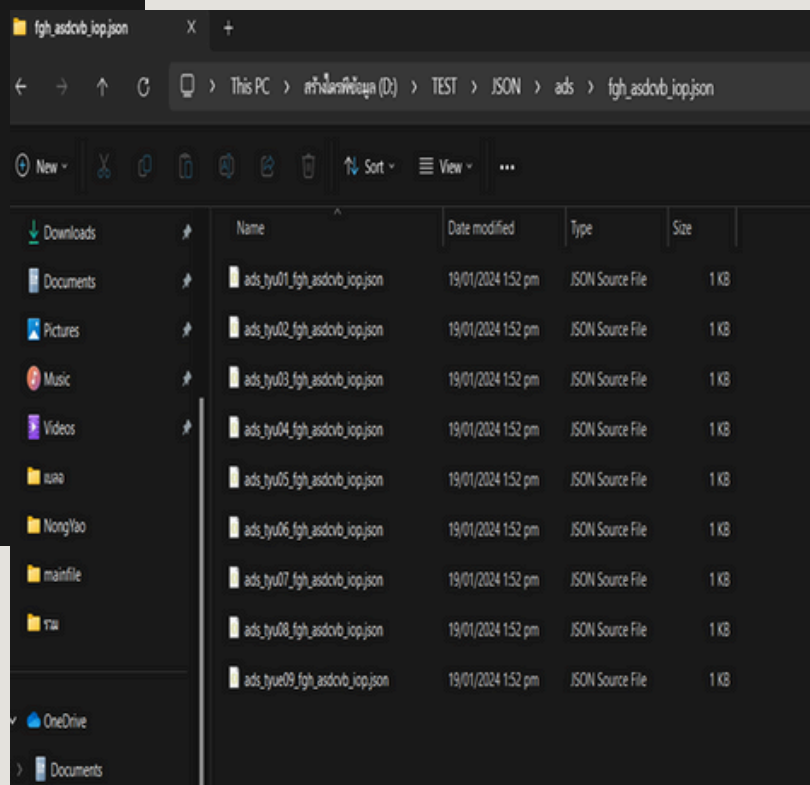
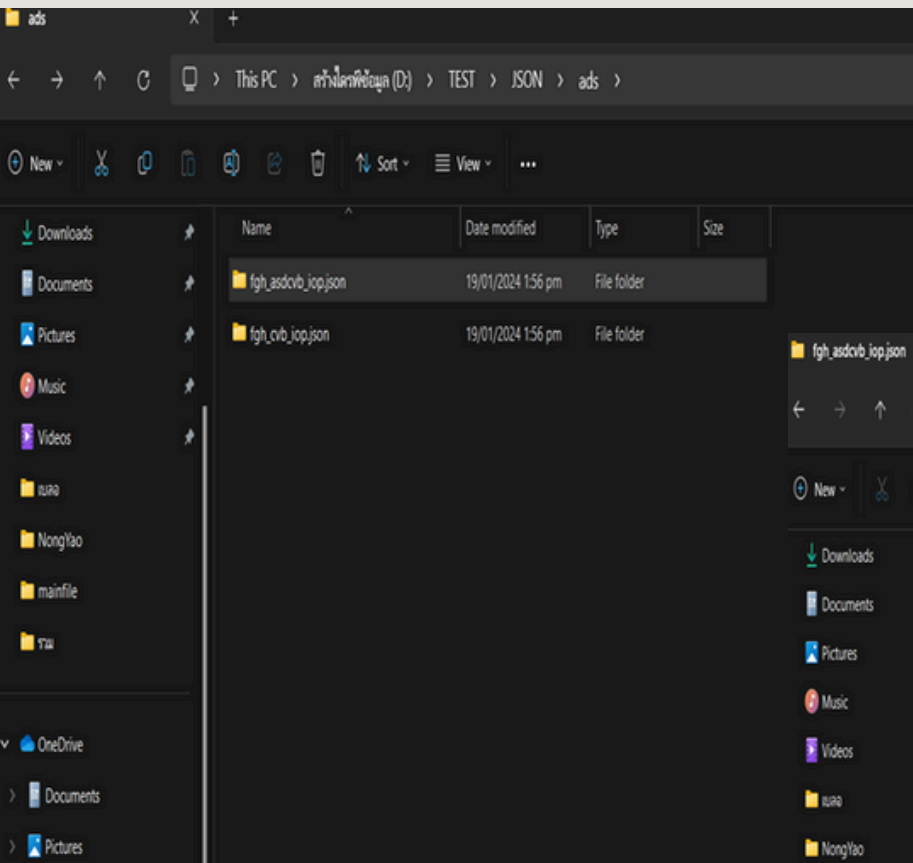
Run Close

Create script for extract Json fromat files with Python



The screenshot shows a Python script named `1.py` in a Visual Studio Code editor. The script is designed to move files from a source directory to a destination directory based on their prefixes. It uses the `os` and `shutil` modules. The `get_destination_folder` function determines the destination path based on the file's prefix. The `move_file` function moves a single file, and the `move_files_to_folders` function iterates through a list of prefixes to move multiple files. The script is set to run in a terminal window.

```
1 import os
2 import shutil
3
4 def get_destination_folder(prefix, subfolder=None):
5     if subfolder:
6         return os.path.join(r'D:\TEST\JSON', prefix, subfolder)
7     else:
8         return os.path.join(r'D:\TEST\JSON', prefix)
9
10 def move_file(source_path, destination_path):
11     shutil.move(source_path, destination_path)
12
13 def move_files_to_folders(folder_path, prefixes):
14     for filename in os.listdir(folder_path):
15         file_path = os.path.join(folder_path, filename)
16         moved = False
17         for prefix in prefixes:
18             if filename.startswith(prefix):
19                 parts = filename.split('.')
20                 destination_folder = get_destination_folder(prefix, "-".join(parts[3:6]))
21                 destination_path = os.path.join(destination_folder, filename)
22                 if not os.path.exists(destination_folder):
23                     os.makedirs(destination_folder)
24                 move_file(file_path, destination_path)
25                 moved = True
26                 break
27         if not moved:
28             parts = filename.split('.')
29             destination_folder = get_destination_folder("others", "-".join(parts[3:]))
30             destination_path = os.path.join(destination_folder, filename)
31             if not os.path.exists(destination_folder):
32                 os.makedirs(destination_folder)
33             move_file(file_path, destination_path)
34
35 folder_path = r'D:\TEST\JSON\others'
36 # Prefixes ที่ต้องย้าย
37 prefixes_to_move = ['ads', 'ert', 'nty']
38
39 move_files_to_folders(folder_path, prefixes_to_move)
```



Association Rule Mining

```
%reset -f
import pandas as pd
import ast
import os
import numpy as np
from scipy import stats
from sqlalchemy import create_engine
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
from dotenv import load_dotenv
load_dotenv()
```

✓ 0.0s

True

```
def parse_braces_list(s):
    if pd.isna(s) or s == '{}':
        return []
    s = s.strip('{}')
    items = s.split(',')
    return [item.strip() for item in items]

def change_type(data_change_type):
    s = data_change_type.strip("[]").split(",")
    arr = [p.strip().strip('"').strip("'") for p in s]
    return arr

def get_menu_types(itemset, df_menu):
    types = []
    for code in itemset:
        match = df_menu.loc[df_menu["menu_code"] == code, "menu_type_code"]
        if not match.empty:
            types.append(match.values[0])
    return list(types)

def menu_type_dummy(all_menu_type, df_menu):
    menu_types = sorted(df_menu["menu_type_code"].unique())
    dummy = {t: 0 for t in menu_types}
    for t in all_menu_type:
        if t in dummy:
            dummy[t] += 1

    return dummy
```

```
def select_data_from_postgress(table_name):
    user = os.getenv("PG_USER")
    password = os.getenv("PG_PASSWORD")
    host = os.getenv("PG_HOST")
    port = os.getenv("PG_PORT")
    database = os.getenv("PG_DATABASE")
    connection_url = f"postgresql+psycopg2://{user}:{password}@{host}:{port}/{database}"
    engine = create_engine(connection_url)
    df = pd.read_sql_table(table_name, con=engine)
    if 'menu_name' in df.columns:
        df['menu_name'] = df['menu_name'].apply(parse_braces_list)
    if 'menu_type' in df.columns:
        df['menu_type'] = df['menu_type'].apply(parse_braces_list)
    return df
```

```
def filter_customer(count_customer):
    table_mapping = {
        1: ['customer_detail_a', 'customer_detail_b', 'customer_detail_c'],
        2: ['customer_detail_d', 'customer_detail_e', 'customer_detail_f', 'customer_detail_g', 'customer_detail_h'],
        3: ['customer_detail_i', 'customer_detail_j', 'customer_detail_k', 'customer_detail_l'],
        4: ['customer_detail_m', 'customer_detail_n'],
        5: ['customer_detail_o']
    }

    data_frames = []
    if count_customer not in table_mapping:
        print(f"ไม่พบบัญชีสำหรับ count_customer = {count_customer}")
        return pd.DataFrame()

    for table_name in table_mapping[count_customer]:
        try:
            df = select_data_from_postgress(table_name)
            if df is not None and not df.empty:
                data_frames.append(df)
        except Exception as e:
            print(f"❌ ผิดข้อมูลจาก {table_name} ไม่พบบัญชี: {e}")

    combined_df = pd.concat(data_frames, ignore_index=True)
    return combined_df
```

```
def outlier_price_interquartile_range(data):
    Q1 = data['price'].quantile(0.10)
    Q3 = data['price'].quantile(0.90)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    data_no_outlier = data[(data['price'] >= lower_bound) & (data['price'] <= upper_bound)].copy()
    return data_no_outlier
```

```

def transactionencoder(data_c):
    transactions = data_c['menu_name'].tolist()
    te = TransactionEncoder()
    te_ary = te.fit(transactions).transform(transactions)
    return pd.DataFrame(te_ary, columns=te.columns_)

def v_support(datatransaction , endcode_menu_name):
    frequent_itemsets = apriori(datatransaction, min_support=0.001, use_colnames=True)
    frequent_itemsets['menu_type'] = frequent_itemsets['itemsets'].apply(lambda x: get_menu_types(x, endcode_menu_name))
    frequent_itemsets['length_itemsets'] = frequent_itemsets['itemsets'].apply(lambda x: len(x))
    frequent_itemsets['length_menu_type'] = frequent_itemsets['menu_type'].apply(lambda x: len(x))
    return frequent_itemsets

def v_association_rules(value_support, endcode_menu_name):
    rules = association_rules(value_support, metric="confidence", min_threshold=0.001)
    rules['antecedents_length'] = rules['antecedents'].apply(lambda x: len(x))
    rules['consequents_length'] = rules['consequents'].apply(lambda x: len(x))
    rules = pd.concat([rules], axis=1)

    return rules.sort_values("support", ascending=False)

```

✓ 0.0s

```

endcode_menu_name = select_data_from_postgress('menu_name_table')

def process_association(table_name, filter_menu = None):
    if isinstance(table_name, int):
        g_data = filter_customer(table_name)
    else:
        g_data = select_data_from_postgress(table_name)
    g_data = g_data[g_data['menu_name'].apply(len) > 1]
    clear_outlier = outlier_price_interquartile_range(g_data)
    c_data = cleansing_data(clear_outlier)
    df_encoded = transactionencoder(c_data)
    frequent_itemsets = v_support(df_encoded , endcode_menu_name)
    rules = v_association_rules(frequent_itemsets, endcode_menu_name)

    if filter_menu :
        filtered = rules[
            rules['antecedents'].apply(lambda x: any(t in x for t in filter_menu)) |
            rules['consequents'].apply(lambda x: any(t in x for t in filter_menu))
        ]
    return filtered

```

return rules

data_association = process_association(3)

✓ 1m 42.6s

data_association

✓ 0.0s

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	representativity	leverage	conviction	zhangs_metric	jaccard	certainty	kulczynski	antecedents_length	consequents_length
539	(l)	(3)	0.115675	0.116223	0.012502	0.108080	0.929936	1.0	-0.000942	0.990870	-0.078509	0.056985	-0.009214	0.107825	1	
538	(3)	(l)	0.116223	0.115675	0.012502	0.107571	0.929936	1.0	-0.000942	0.990918	-0.078554	0.056985	-0.009165	0.107825	1	
1169	(f)	(u)	0.113328	0.118716	0.012288	0.108432	0.913367	1.0	-0.001166	0.988464	-0.096636	0.055918	-0.011670	0.105971	1	
1168	(u)	(f)	0.118716	0.113328	0.012288	0.103510	0.913367	1.0	-0.001166	0.989048	-0.097169	0.055918	-0.011073	0.105971	1	
907	(y)	(9)	0.113662	0.117960	0.012247	0.107746	0.913411	1.0	-0.001161	0.988553	-0.096620	0.055825	-0.011580	0.105783	1	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
22835	(a)	(g, c)	0.113365	0.011605	0.001001	0.008834	0.761189	1.0	-0.000314	0.997204	-0.261365	0.008078	-0.002804	0.047563	1	
22836	(c, i)	(a)	0.011657	0.113365	0.001001	0.085906	0.757783	1.0	-0.000320	0.969961	-0.244375	0.008075	-0.030970	0.047370	2	
22837	(c, a)	(i)	0.011730	0.112322	0.001001	0.085371	0.760060	1.0	-0.000316	0.970534	-0.242098	0.008138	-0.030361	0.047143	2	
22838	(i, a)	(c)	0.011454	0.115331	0.001001	0.087432	0.758092	1.0	-0.000320	0.969427	-0.244027	0.007962	-0.031537	0.048057	2	
15783	(7)	(4, e)	0.116823	0.011923	0.001001	0.008572	0.718947	1.0	-0.000391	0.996620	-0.306823	0.007839	-0.003392	0.046281	1	

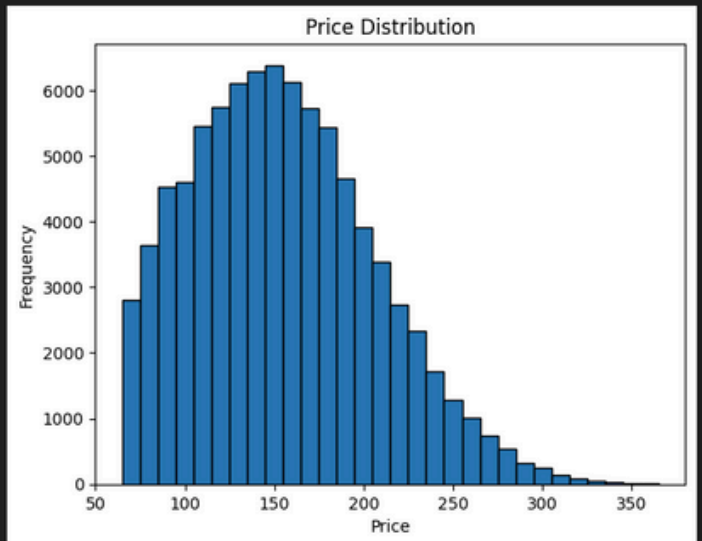
30450 rows x 16 columns

```
import matplotlib.pyplot as plt
```

```

plt.hist(data_set_a['price'], bins=30, edgecolor='k')
plt.title('Price Distribution')
plt.xlabel('Price')
plt.ylabel('Frequency')
plt.show()

```



Project

การทำนายราคาทองคำด้วยการเรียนรู้ของเครื่องบนอนุกรมเวลาแบบแบน

วัตถุประสงค์

เพื่อออกแบบ พัฒนา และทดสอบแบบจำลองราคาทองคำรายวัน และวิเคราะห์ประเมินประสิทธิภาพของแบบจำลอง ตลอดจนเปรียบเทียบประสิทธิภาพของแบบจำลองการทำนาย ราคาทองคำแบบต่างๆ โดยใช้ข้อมูลย้อนหลัง 10 ปี จากเว็บทองคำ.com หรือ สมาคมค้าทองคำ และจะแบ่งข้อมูลเป็น 3 ส่วน เป็น 7 วัน 14 วัน และ 30 วันย้อนหลัง

อัลกอริทึมที่ใช้

- Decision Tree Regression
- Random Forest Regression
- Artificial Neural Network
- Linear Regression
- Lasso Regression

เครื่องมือในการดำเนินโครงการ

Microsoft Excel , Python , Google Colab
มีการใช้ไลบรารีจาก Scikit - Learn และ TensorFlow

ทดสอบโมเดลด้วย

MAE , MSE , RMSE และโมเดล Baseline และสร้างแบบจำลองการซื้อขายเพื่อดูกันว่าโมเดลที่ได้ค่า MAE , MSE , RMSE น้อยที่สุดหรือดีที่สุดจะได้กำไรหรือขาดทุน

```

df = pd.read_excel('/content/drive/MyDrive/Project/plt.xlsx')
x123 = df.iloc[:, 1]
x123_normalized = (x123 - x123.min()) / (x123.max() - x123.min())
USD = df.iloc[:, 2]
USD_normalized = (USD - USD.min()) / (USD.max() - USD.min())

plt.scatter(x123_normalized, USD_normalized, c=USD_normalized, label='USD')
plt.xlabel("Goldspot Data (Normalized)")
plt.ylabel("USD Data (Normalized)")
plt.title("Scatter Plot of Normalized Goldspot vs Normalized USD")
plt.colorbar(label='USD (Normalized)')
plt.legend()
plt.show()

GOLDSPOT = df.iloc[:, 1]
DATE = df.iloc[:, 0]
plt.figure(figsize=(20, 6))
plt.plot(DATE, GOLDSPOT)
plt.grid(True)
plt.xlabel("year")
plt.ylabel("gold spot")
plt.show()

# sns.pairplot(df)

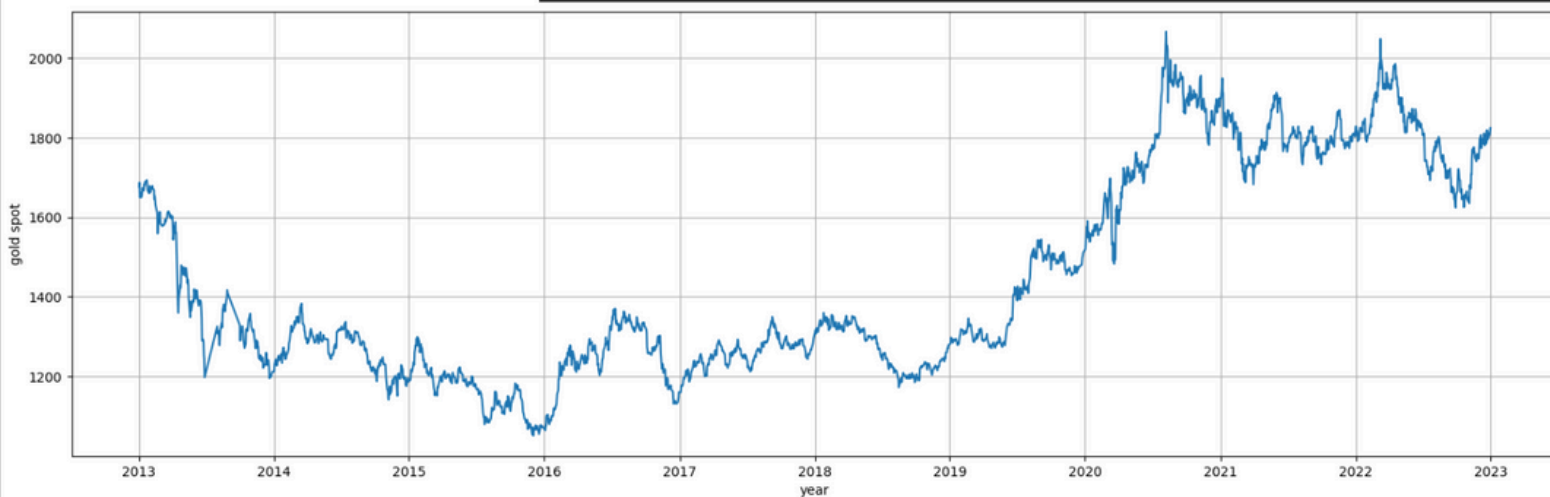
# train
data_train = pd.read_excel('/content/drive/MyDrive/Project/PDATA_Flannten_Time Series.xlsx', '7DAY')

# test
data_test = pd.read_excel('/content/drive/MyDrive/Project/PDATA_Flannten_Time Series.xlsx', 'DATA TEST 7DAY')

# train
train_target = data_train.iloc[:, 1] # Y
train_factor = data_train.iloc[:, 4:] # x

# test
test_target = data_test.iloc[:, 1] # Y
test_factor = data_test.iloc[:, 4:] # x

```



```

plt.figure(figsize=(15, 6))
plt.plot(test_target.values.flatten(), label='test_target')
plt.plot(predictions.flatten(), label='predictions')
plt.legend()
plt.show()

```

```

RES=test_target - predictions
sns.displot(RES,bins=30, kde=True)

```

```

lasso_model = Lasso(positive=True)
lasso_model.fit(train_factor, train_target)
predictions = lasso_model.predict(test_factor)
predictions = np.round(predictions, 2)

date = data_test.iloc[:, 0]

for idx, (d, target, pred) in enumerate(zip(date, test_target, predictions), start=1):
    date_str = d.strftime("%Y-%m-%d")
    # print(f"Row {idx}: Date:{date_str}, Actual: {target}, Predicted: {pred}")

mse = mean_squared_error(test_target, predictions)
print(f"MSE: {mse}")

mae = mean_absolute_error(test_target, predictions)
print(f"MAE: {mae}")

rmse = mean_squared_error(test_target, predictions, squared=False)
print(f"RMSE: {rmse}")

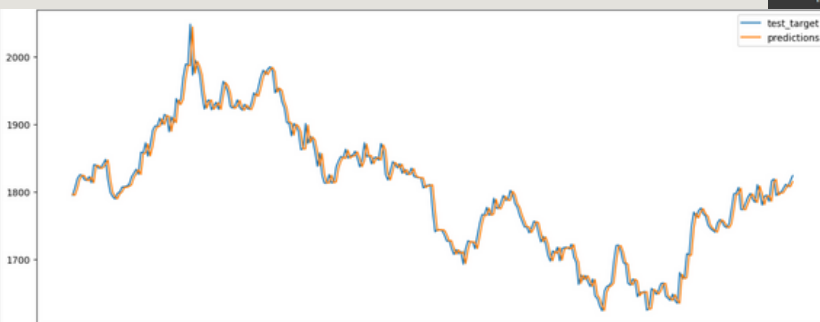
r_squared = r2_score(test_target, predictions)
print(f"R-squared: {r_squared}")

```

```

MSE: 223.2899517915311
MAE: 11.005667752443001
RMSE: 14.942889673404242

```



แบบจำลองการซื้อขาย

```

missed_opportunities_count = 0
no_trade_count = 0
buy_count = 0
profitable_buy_count = 0
loss_buy_count = 0
zero_profit_buy_count = 0
buy_sell_indicator_zero_count = 0

predictions = pd.DataFrame({'Predictions': predictions})
predictions = predictions.shift(periods=-1)
predictions = np.array(predictions)
predictions
Actual1 = test_target
Actual2 = test_target.shift(-1)

buy_sell = []
profits = []

for idx, (d, actual1, actual2, predictions) in enumerate(zip(date, Actual1, Actual2, predictions), start=1):
    date_str = d.strftime('%Y-%m-%d')

    if actual1 < predictions:
        buy_sell_indicator = 1
        profit = actual2 - actual1
        action_description = "ซื้อ"
    else:
        if actual1 < actual2:
            buy_sell_indicator = 0
            profit = 0
            action_description = "เสียโอกาส"
        else:
            buy_sell_indicator = 0
            profit = 0
            action_description = "ไม่ทำการซื้อขาย"

    buy_sell.append(buy_sell_indicator)
    profits.append(profit)

    if action_description == "เสียโอกาส":
        missed_opportunities_count += 1
    elif action_description == "ไม่ทำการซื้อขาย":
        no_trade_count += 1
    elif action_description == "ซื้อ":
        buy_count += 1
        if profit > 0:
            profitable_buy_count += 1
        elif profit < 0:
            loss_buy_count += 1
        if profit == 0:
            zero_profit_buy_count += 1

```

Row 290: Date: 2022-12-12,	Actual 1: 1789.0,	Actual 2: 1784.5,	Predictions tomorrow: [1789.16],	Buy/Sell: 1,	Profit: -4.5,	Action: ซื้อ
Row 291: Date: 2022-12-13,	Actual 1: 1784.5,	Actual 2: 1810.5,	Predictions tomorrow: [1784.31],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 292: Date: 2022-12-14,	Actual 1: 1810.5,	Actual 2: 1797.0,	Predictions tomorrow: [1808.72],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย
Row 293: Date: 2022-12-15,	Actual 1: 1797.0,	Actual 2: 1780.5,	Predictions tomorrow: [1797.19],	Buy/Sell: 1,	Profit: -16.5,	Action: ซื้อ
Row 294: Date: 2022-12-16,	Actual 1: 1780.5,	Actual 2: 1793.5,	Predictions tomorrow: [1781.],	Buy/Sell: 1,	Profit: 13.0,	Action: ซื้อ
Row 295: Date: 2022-12-17,	Actual 1: 1793.5,	Actual 2: 1795.0,	Predictions tomorrow: [1792.59],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 296: Date: 2022-12-19,	Actual 1: 1795.0,	Actual 2: 1786.0,	Predictions tomorrow: [1794.55],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย
Row 297: Date: 2022-12-20,	Actual 1: 1786.0,	Actual 2: 1816.0,	Predictions tomorrow: [1786.07],	Buy/Sell: 1,	Profit: 30.0,	Action: ซื้อ
Row 298: Date: 2022-12-21,	Actual 1: 1816.0,	Actual 2: 1819.0,	Predictions tomorrow: [1814.22],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 299: Date: 2022-12-22,	Actual 1: 1819.0,	Actual 2: 1794.5,	Predictions tomorrow: [1818.36],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย
Row 300: Date: 2022-12-23,	Actual 1: 1794.5,	Actual 2: 1799.0,	Predictions tomorrow: [1795.19],	Buy/Sell: 1,	Profit: 4.5,	Action: ซื้อ
Row 301: Date: 2022-12-24,	Actual 1: 1799.0,	Actual 2: 1799.0,	Predictions tomorrow: [1798.41],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย
Row 302: Date: 2022-12-26,	Actual 1: 1799.0,	Actual 2: 1805.0,	Predictions tomorrow: [1798.63],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 303: Date: 2022-12-27,	Actual 1: 1805.0,	Actual 2: 1811.0,	Predictions tomorrow: [1804.25],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 304: Date: 2022-12-28,	Actual 1: 1811.0,	Actual 2: 1807.5,	Predictions tomorrow: [1810.39],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย
Row 305: Date: 2022-12-29,	Actual 1: 1807.5,	Actual 2: 1816.0,	Predictions tomorrow: [1807.37],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 306: Date: 2022-12-30,	Actual 1: 1816.0,	Actual 2: 1824.0,	Predictions tomorrow: [1815.11],	Buy/Sell: 0,	Profit: 0,	Action: เสียโอกาส
Row 307: Date: 2022-12-31,	Actual 1: 1824.0,	Actual 2: nan,	Predictions tomorrow: [nan],	Buy/Sell: 0,	Profit: 0,	Action: ไม่ทำการซื้อขาย

ผลรวมของกำไร: 73.0

จำนวนการซื้อ: 96

จำนวนการซื้อกำไร: 47

จำนวนการซื้อขาดทุน: 46

จำนวนการซื้อแต่ไม่ได้กำไร 3

จำนวนครั้งที่ไม่ซื้อ: 211

จำนวนครั้งที่ เสียโอกาส: 100

จำนวนครั้งที่ไม่ทำการซื้อขาย: 111

Web Scraping Python

```
import requests
from bs4 import BeautifulSoup
import pandas as pd

# ดึงข้อมูลจากเว็บไซต์
response = requests.get("https://www.myhora.com/%E0%B8%AB%E0%B8%A7%E0%B8%A2/%E0%B8%AA%E0%B8%
soup = BeautifulSoup(response.text, 'html.parser')

# ดึงข้อมูลที่ต้องการจากเว็บไซต์
data = []
temp_row = []
a = soup.find_all('div', class_='colx')
del a[:28]

for i, div in enumerate(a, start=1):
    temp_row.append(div.text.strip())
    if i % 12 == 0:
        data.append(temp_row)
        temp_row = []

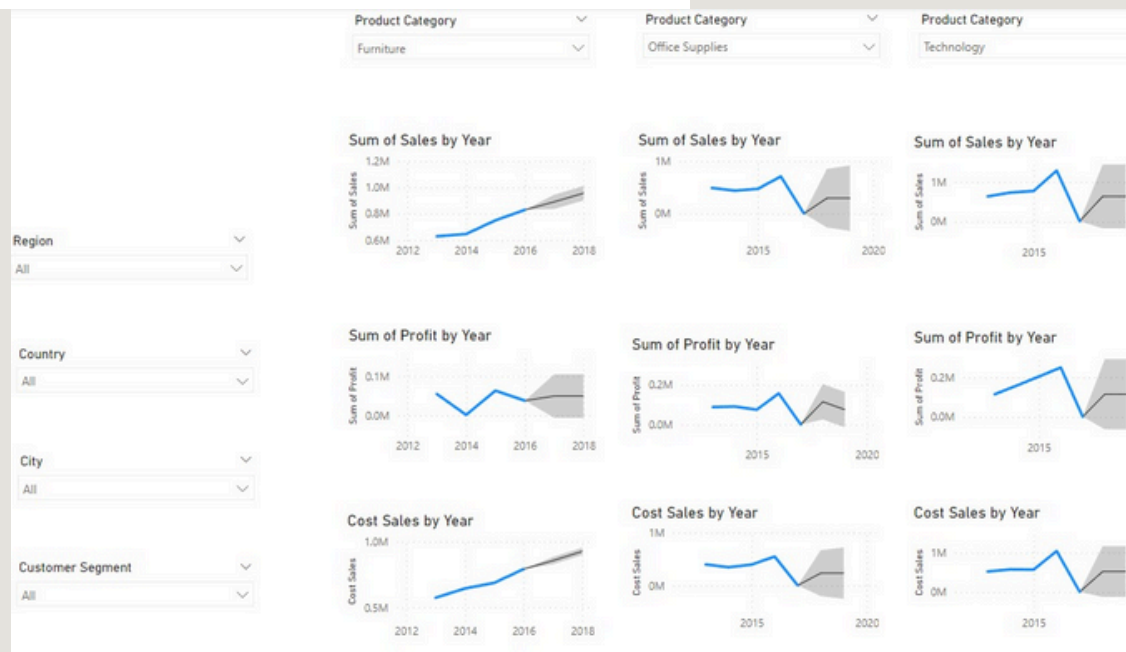
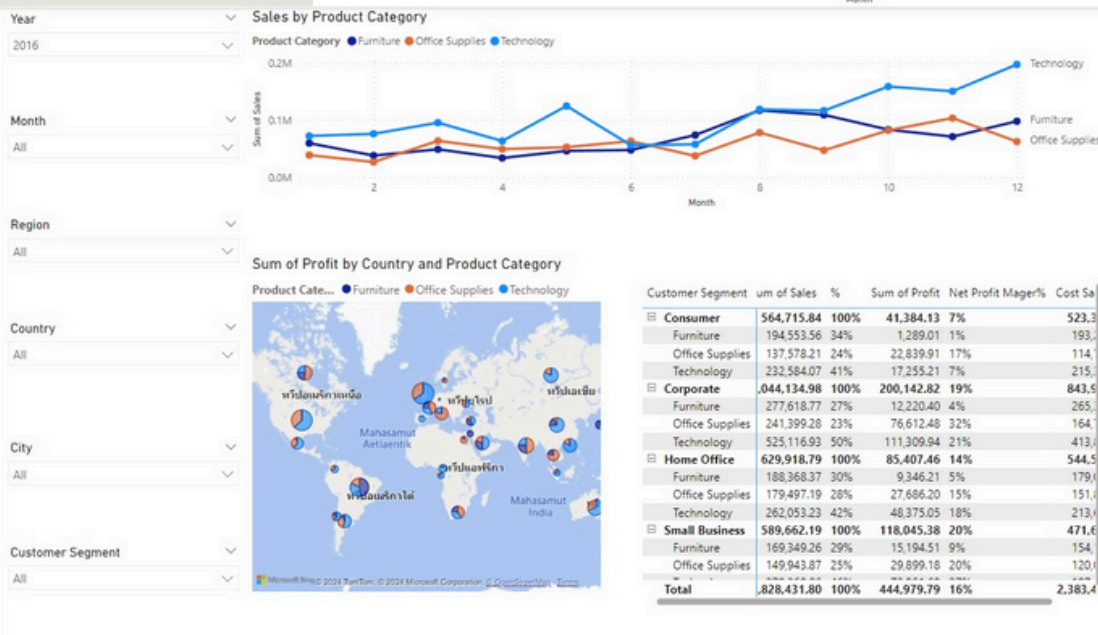
if temp_row:
    data.append(temp_row)

df = pd.DataFrame(data)

df.to_excel("หอย.xlsx", index=False, header=False)
```

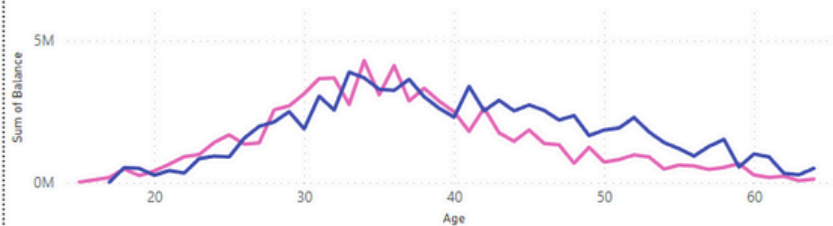
	A	B	C	D	E	F	G	H	I	J	K	L	M
1	1	เมษายน	เม.ย.	2567	67	803481	81	481	90	122 809 5 90		122 809 559 947	
2	16	มีนาคม	มี.ค.	2567	67	997626	26	626	78	509 571 3 78		509 571 329 794	
3	1	มีนาคม	มี.ค.	2567	67	253603	03	603	79	900 975 3 79		900 975 382 703	
4	16	กุมภาพันธ์	ก.พ.	2567	67	941395	95	395	43	056 330 3 43		056 330 375 587	
5	1	กุมภาพันธ์	ก.พ.	2567	67	607063	63	063	09	454 943 5 09		454 943 544 591	
6	17	มกราคม	ม.ค.	2567	67	105979	79	979	61	429 931 1 61		429 931 196 635	
7	30	ธันวาคม	ธ.ค.	2566	66	625544	44	544	89	600 648 4 89		600 648 456 882	
8	16	ธันวาคม	ธ.ค.	2566	66	356757	57	757	85	058 410 5 85		058 410 584 964	
9	1	ธันวาคม	ธ.ค.	2566	66	251097	97	097	91	055 265 0 91		055 265 092 280	
10	16	พฤศจิกายน	พ.ย.	2566	66	557990	90	990	14	346 412 7 14		346 412 778 961	
11	1	พฤศจิกายน	พ.ย.	2566	66	743951	51	951	63	335 913 0 63		335 913 019 349	
12	16	ตุลาคม	ต.ค.	2566	66	931446	46	446	44	167 398 2 44		167 398 272 970	
13	1	ตุลาคม	ต.ค.	2566	66	727202	02	202	66	324 355 4 66		324 355 426 615	
14	16	กันยายน	ก.ย.	2566	66	320812	12	812	46	037 699 0 46		037 699 057 344	
15	1	กันยายน	ก.ย.	2566	66	915478	78	478	91	521 596 2 91		521 596 291 692	
16	16	สิงหาคม	ส.ค.	2566	66	471782	82	782	67	431 739 7 67		431 739 737 742	
17	31	กรกฎาคม	ก.ค.	2566	66	260453	53	453	11	268 708 3 11		268 708 387 601	
18	16	กรกฎาคม	ก.ค.	2566	66	169530	30	530	62	261 384 0 62		261 384 066 780	
19	1	กรกฎาคม	ก.ค.	2566	66	922605	05	605	16	281 867 4 16		281 867 491 947	
20	16	มิถุนายน	มิ.ย.	2566	66	264872	72	872	30	519 628 2 30		519 628 202 874	
21	1	มิถุนายน	มิ.ย.	2566	66	125272	72	272	09	000 681 3 09		000 681 386 971	
22	16	พฤษภาคม	พ.ค.	2566	66	132903	03	903	99	678 739 6 99		678 739 693 731	
23	2	พฤษภาคม	พ.ค.	2566	66	843019	19	019	65	500 780 1 65		500 780 187 269	
24	16	เมษายน	เม.ย.	2566	66	984906	06	906	71	670 678 5 71		670 678 551 797	
25	1	เมษายน	เม.ย.	2566	66	087907	07	907	99	111 914 2 99		111 914 290 698	
26	16	มีนาคม	มี.ค.	2566	66	025873	73	873	73	420 800 3 73		420 800 355 544	
27	1	มีนาคม	มี.ค.	2566	66	417652	52	652	55	577 919 7 55		577 919 748 984	
28	16	กุมภาพันธ์	ก.พ.	2566	66	590417	17	417	80	195 664 3 80		195 664 377 523	
29	1	กุมภาพันธ์	ก.พ.	2566	66	297411	11	411	92	181 789 1 92		181 789 101 664	
30	17	มกราคม	ม.ค.	2566	66	812519	19	519	47	389 443 5 47		389 443 564 849	
31	30	ธันวาคม	ธ.ค.	2565	65	157196	96	196	58	007 522 2 58		007 522 250 425	
32	16	ธันวาคม	ธ.ค.	2565	65	845093	93	093	14	411 912 5 14		411 912 593 855	
33	1	ธันวาคม	ธ.ค.	2565	65	375805	05	805	08	170 786 4 08		170 786 409 421	
34	16	พฤศจิกายน	พ.ย.	2565	65	121789	89	789	64	532 722 1 64		532 722 157 973	
35	1	พฤศจิกายน	พ.ย.	2565	65	913106	06	106	70	722 839 3 70		722 839 343 922	
36	16	ตุลาคม	ต.ค.	2565	65	613106	06	106	15	037 158 6 15		037 158 606 799	

Data Visualization

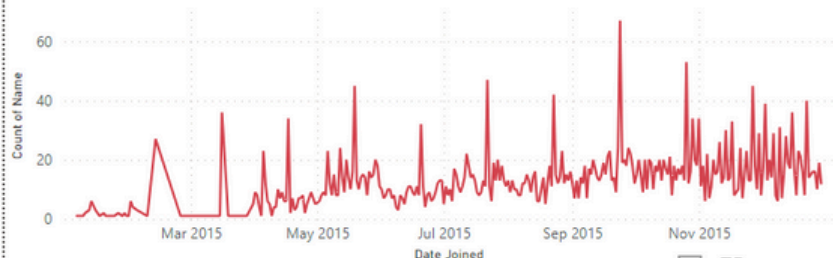


Balance by Age and Gender

Gender ● Female ● Male

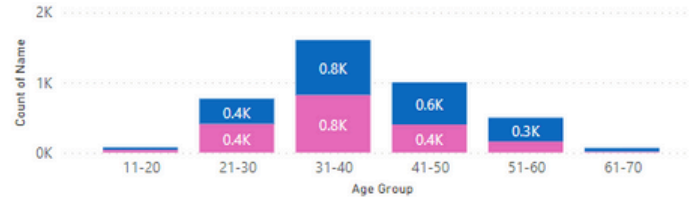


Total Customer

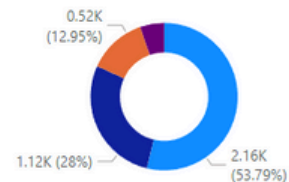


Number of Customer by Age and Gender

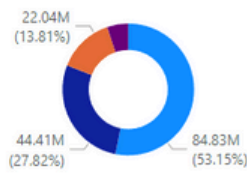
Gender ● Female ● Male



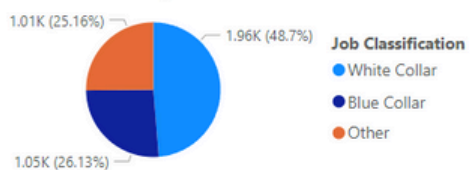
Count of Name by Region



Balance by Region

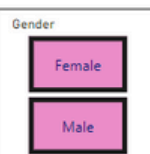


Number Customer by Job



- Region
- ☐ England
 - ☐ Northern Ireland
 - ☐ Scotland
 - ☐ Wales

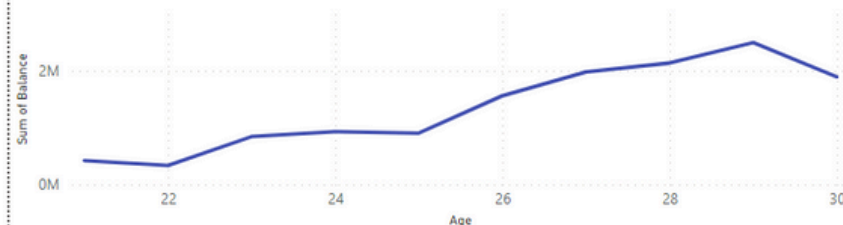
4014
Count of Name



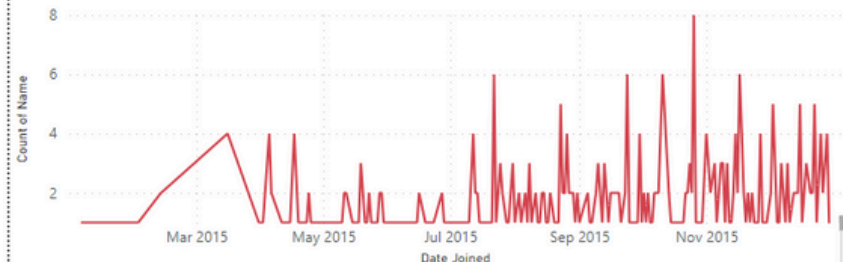
Bank Data Analytics

Balance by Age and Gender

Gender ● Male

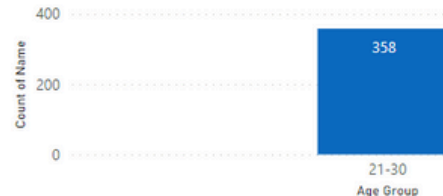


Total Customer

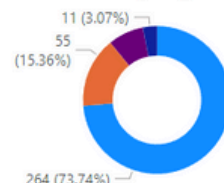


Number of Customer by Age and Gender

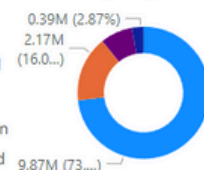
Gender ● Male



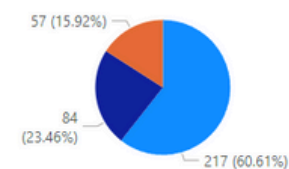
Count of Name by Region



Balance by Region

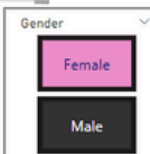


Number Customer by Job



- Region
- ☐ England
 - ☐ Northern Ireland
 - ☐ Scotland
 - ☐ Wales

358
Count of Name



Bank Data Analytics

Certificate

OF COMPLETION

ກູດລ ວຽນຸຕຣ

has successfully completed

Get started with Programming ຮວມສິ່ງທີ່ຕ້ອງຮູ້ ກ່ອນເດີນສູ່ເສັ້ນທາງໂປຣແກຣມເມວ໌

an online course offered by FutureSkill

03/04/2024



FutureSkill has confirmed the identity of this individual and their participation in the course

Futureskill.co

Certificate

OF COMPLETION

ກູດລ ວຽນຸຕຣ

has successfully completed

ຈັດການຂໍ້ມູນລູກຢ່າງມີປະສິດທິພາບ ພ້ອມກັບວິເຄາະຂໍ້ມູນທີ່ມີໃນໄມ້ທີ່ອັດໄຈດ້ວຍ SQL Command

an online course offered by FutureSkill

19/12/2023



FutureSkill has confirmed the identity of this individual and their participation in the course

Futureskill.co

Certificate

OF COMPLETION

ກູດລ ວຽນຸຕຣ

has successfully completed

UpSkill Python Programming ເລີ່ມທາງສູ່ Developer

an online course offered by FutureSkill

11/01/2024



FutureSkill has confirmed the identity of this individual and their participation in the course

Futureskill.co

ภูวดล วรรณตร

ได้ผ่านการเรียนออนไลน์ตามเกณฑ์การวัดผลในรายวิชา

ความรู้พื้นฐานเกี่ยวกับบักดาตา (10 ชั่วโมงการเรียนรู้)

พัฒนารายวิชาโดย มหาวิทยาลัยสุโขทัยธรรมาธิราช

ผศ.ดร. วุฒิชัย ร่มสายหยุด
สาขาวิชาวิทยาศาสตร์และเทคโนโลยี
มหาวิทยาลัยสุโขทัยธรรมาธิราช

ภูวดล วรรณตร

ได้ผ่านการเรียนออนไลน์ตามเกณฑ์การวัดผลในรายวิชา

วิทยาการข้อมูลและการประยุกต์ใช้ (30 ชั่วโมงการเรียนรู้)

พัฒนารายวิชาโดย จุฬาลงกรณ์มหาวิทยาลัย

ผศ.ดร.ณัฐวุฒิ หนูโพธิ์วงษ์
วิทยาศาสตร์จารย์
ศูนย์ จุฬาลงกรณ์มหาวิทยาลัย

ภูวดล วรรณตร

ได้ผ่านการเรียนออนไลน์ตามเกณฑ์การวัดผลในรายวิชา

การวิเคราะห์เชิงสถิติกับมหาข้อมูล (10 ชั่วโมงการเรียนรู้)

พัฒนารายวิชาโดย มหาวิทยาลัยรังสิต

ผศ. ดร. เชษฐพงศ์ ศรีสุอาน
คณะบดี, ว. นวัตกรรมดิจิทัลและเทคโนโลยีสารสนเทศ
มหาวิทยาลัยรังสิต

ภูวดล วรรณตร

ได้ผ่านการเรียนออนไลน์ตามเกณฑ์การวัดผลในรายวิชา

ภาษาอังกฤษสำหรับเทคโนโลยีสารสนเทศ (10 ชั่วโมงการเรียนรู้)

พัฒนารายวิชาโดย มหาวิทยาลัยขอนแก่น

ผศ.ดร.กรวิภา พูลผล
รองคณบดีฝ่ายแผนและสารสนเทศ
คณะมนุษยศาสตร์และสังคมศาสตร์ มหาวิทยาลัยขอนแก่น



หลักสูตรวิทยาการการจัดการข้อมูล
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ
ขอมอบเกียรติบัตรฉบับนี้ให้ไว้เพื่อแสดงว่า

นาย ภูวดล วรบุตร

ผ่านการอบรม หัวข้อ

Google Data Studio กับการวิเคราะห์ข้อมูลเชิงธุรกิจ

โครงการพัฒนาสมรรถนะวิชาชีพนักศึกษาประจำหลักสูตร

วันที่ ๑๐ กุมภาพันธ์ พ.ศ. ๒๕๖๖

(ดร.จุริรา คงนุ้ย)

ประธานหลักสูตรวิทยาการการจัดการข้อมูล

Made for free with Certify'em



สำนักวิทยบริการและเทคโนโลยีสารสนเทศ
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ

ขอมอบวุฒิบัตรให้ไว้เพื่อแสดงว่า

นาย ภูวดล วรบุตร

เข้าร่วมอบรมแนะนำการใช้งาน

Google Workspace for Education Plus

ให้ไว้ ณ วันที่ 30 มกราคม 2566

ดร.เอกชัย เนาวนิช

ผู้อำนวยการสำนักวิทยบริการและเทคโนโลยีสารสนเทศ



สำนักวิทยบริการและเทคโนโลยีสารสนเทศ
มหาวิทยาลัยเทคโนโลยีราชมงคลสุวรรณภูมิ

ขอมอบวุฒิบัตรให้ไว้เพื่อแสดงว่า

นาย ภูวดล วรบุตร

ได้เข้าร่วมเสริมสร้างความฉลาดรู้ด้านดิจิทัลในอาเซียน

: ASEAN DIGITAL LITERACY PROGRAMME (ADLP) จำนวน 3 ชม.

ให้ไว้ ณ วันที่ 25 มกราคม พ.ศ. 2566

นายเอกชัย เนาวนิช

ผู้อำนวยการสำนักวิทยบริการและเทคโนโลยีสารสนเทศ



thank you